# Hybrid Cryptography in TLS 1.3
# Performance Analysis

Brynja Eyfjörð Jónsdóttir
s221901

Karítas Ósk Pálmadóttir
s175414

Kristofer Birgir Hjörleifsson
s253156

March 17, 2026

**Abstract**

This project investigates the practical use of hybrid cryptography and evaluates its impact on the performance of TLS 1.3 connections. Hybrid cryptography combines classical and post-quantum cryptographic algorithms in order to maintain security against both classical and quantum adversaries.

In this report we review ETSI guidance on hybrid cryptographic constructions and summarize ECCG/ENISA recommendations for quantum-secure algorithms within the European Union. We then implement a TLS client and server using OpenSSL and benchmark the performance of classical TLS compared with hybrid TLS using post-quantum key encapsulation mechanisms. The experiments measure handshake time, data transfer time, and communication overhead introduced by hybrid cryptography.

## 1 Introduction

The development of quantum computers poses a potential threat to widely deployed cryptographic algorithms such as RSA and elliptic-curve cryptography. Algorithms based on integer factorization and discrete logarithms may be broken by quantum algorithms such as Shor's algorithm.

To address this risk, new cryptographic schemes designed to resist quantum attacks have been standardized. However, because these schemes rely on mathematical assumptions that are not yet as extensively studied as classical cryptographic constructions, organizations such as ECCG and ENISA recommend hybrid deployment during the transition to post-quantum cryptography.

Hybrid cryptography combines classical and post-quantum algorithms so that security is preserved as long as at least one of the algorithms remains secure.

This project investigates how hybrid cryptography can be applied in practice and evaluates its impact on the performance of TLS 1.3 connections.

## 2 Background

### 2.1 Transport Layer Security (TLS)

Transport Layer Security (TLS) is a widely used protocol for establishing secure communication channels over the Internet. TLS provides:

- Confidentiality
- Integrity

- Authentication

TLS 1.3 performs a cryptographic handshake to establish shared session keys between a client and server.

### 2.2 Classical Cryptography

Traditional TLS deployments rely on classical cryptographic primitives such as:

- RSA
- Diffie–Hellman
- Elliptic Curve Diffie–Hellman (ECDHE)

These primitives rely on mathematical problems believed to be difficult for classical computers.

### 2.3 Post-Quantum Cryptography

Post-quantum cryptography refers to cryptographic algorithms believed to be secure against attacks from quantum computers.

Examples of standardized algorithms include:

- ML-KEM (Kyber)
- FrodoKEM
- ML-DSA (Dilithium)
- SLH-DSA (SPHINCS+)

### 2.4 Hybrid Cryptography

Hybrid cryptography combines classical and post-quantum algorithms within a single cryptographic protocol. This approach provides security as long as either algorithm remains secure.

## 3 ETSI Hybrid Cryptographic Constructions

The European Telecommunications Standards Institute (ETSI) has published guidance on how to combine classical and post-quantum cryptographic algorithms in hybrid constructions [1]. The document discusses deployment considerations for achieving quantum resistance while retaining the well-studied security guarantees of classical algorithms during the transition period.

### 3.1 Hybrid Key Encapsulation Mechanisms

The following constructions summarize the approaches discussed in ETSI TR 103 966 for combining a classical KEM (e.g., ECDH-based) with a post-quantum KEM (e.g., ML-KEM):

**Concatenation Only**

Both classical and post-quantum KEMs are executed independently and the resulting shared secrets are concatenated directly:

$$K = K_{\text{classical}} \parallel K_{\text{pq}}$$

This approach is the simplest to implement but may not provide strong security guarantees against active attacks, as no additional binding between the components is enforced.

**Concatenation with Simple KDF**

The concatenated shared secrets are passed through a key derivation function (KDF):

$$K = \mathrm{KDF}(K_{\text{classical}} \parallel K_{\text{pq}})$$

This improves security over raw concatenation by ensuring the combined key is well-distributed, but still depends on careful design of the KDF inputs.

**Concatenation with Full KDF**

A full KDF is used that incorporates additional context such as the ciphertexts exchanged during the protocol:

$$K = \mathrm{KDF}(K_{\text{classical}} \parallel K_{\text{pq}} \parallel C_{\text{classical}} \parallel C_{\text{pq}})$$

Including ciphertexts strengthens the binding between the two component mechanisms and gives stronger security properties. This approach aligns with the hybrid key exchange constructions described for TLS in ETSI and IETF documents.

## 3.2 Hybrid Digital Signatures

For digital signatures, ETSI TR 103 966 discusses the following hybrid approaches:

**Concatenation Only (Separable)**

Both a classical signature (e.g., ECDSA) and a post-quantum signature (e.g., ML-DSA) are generated independently and concatenated:

$$\sigma = \sigma_{\text{classical}} \parallel \sigma_{\text{pq}}$$

A verifier checks both signatures separately. This construction is separable, meaning either signature can be stripped and verified on its own.

**Concatenation with Identifiers (Weak Non-Separability)**

Each component signs a message that includes algorithm identifiers, binding the construction to the intended algorithms:

$$\sigma = (ID_1, \sigma_{\text{classical}}) \parallel (ID_2, \sigma_{\text{pq}})$$

This achieves weak non-separability: an attacker cannot easily replace one component with a different algorithm without detection.

**Strong Non-Separability**

The signatures are tightly bound such that neither component can be removed or replaced without invalidating the result. This provides the strongest protection against downgrade attacks, where an attacker might strip the post-quantum component to force purely classical verification.

## 3.3 Advantages and Disadvantages

The TLS hybrid key exchange groups defined by the IETF, including X25519MLKEM768 which we use in this project, are consistent with the concatenation with full KDF approach [8]. The two key shares are sent independently during the TLS handshake, and the resulting shared secrets are combined using the TLS 1.3 key schedule (HKDF), which acts as the key combiner.

| Approach | Advantages | Disadvantages |
|---|---|---|
| Concat. Only (KEM) | Simplest to implement; no changes to underlying algorithms | No binding between components; weaker security guarantees against active attacks |
| Concat. + Full KDF | Strong security properties; incorporates ciphertexts as context | Slightly more complex; KDF design must be carefully specified |
| Separable Signatures | Simple; both signatures independently verifiable | Large combined size; either signature can be stripped |
| Strongly Non-Separable Signatures | Resistant to downgrade and stripping attacks | More complex verification logic; not yet widely supported |

Table 1: Comparison of hybrid cryptographic constructions as described in ETSI TR 103 966.

# 4 ECCG / ENISA Recommendations

The European Union Agency for Cybersecurity (ENISA) publishes the *Agreed Cryptographic Mechanisms* document through the European Cybersecurity Certification Group (ECCG) [2]. This document provides recommendations for cryptographic algorithms and parameter levels that are considered acceptable for use within the EU, including guidance on the transition to quantum-secure cryptography.

## 4.1 Recommended Post-Quantum Algorithms

The ECCG document identifies the following post-quantum algorithms as agreed cryptographic mechanisms in the EU:

### Key Encapsulation Mechanisms

- **ML-KEM** (Module-Lattice-Based Key Encapsulation Mechanism, formerly Kyber) – standardized by NIST in FIPS 203. ML-KEM is based on the hardness of the Module Learning With Errors (MLWE) problem. It is an agreed key encapsulation mechanism, widely adopted and standardized by NIST.
- **FrodoKEM** – a lattice-based KEM based on the plain Learning With Errors (LWE) problem. FrodoKEM uses unstructured lattices, which provide more conservative security assumptions than the module lattices used by ML-KEM, at the cost of larger key sizes and ciphertexts.

### Digital Signature Schemes

- **ML-DSA** (Module-Lattice-Based Digital Signature Algorithm, formerly Dilithium) – standardized in FIPS 204. Based on the MLWE and Module-SIS problems. It is an agreed digital signature scheme, standardized by NIST.
- **SLH-DSA** (Stateless Hash-Based Digital Signature Algorithm, formerly SPHINCS+) – standardized in FIPS 205. Based on hash functions only, providing a conservative security assumption. However, signatures are significantly larger than ML-DSA.
- **XMSS and LMS** – stateful hash-based signature schemes. These provide strong security guarantees based on hash functions, but require careful state management to avoid key reuse.

The ECCG document distinguishes between algorithms that require hybrid deployment and those that may be used standalone. Lattice-based mechanisms such as ML-KEM and ML-DSA should not be used as standalone solutions for their intended security functionality, but instead combined with classical mechanisms in a hybrid construction. For hash-based signature schemes such as SLH-DSA, XMSS, and LMS, standalone use is acceptable, while hybridization is optional for additional assurance [2].

### 4.2 Recommended Security Levels

NIST defines five security levels for post-quantum algorithms. The ECCG document recommends the use of higher parameter sets, as shown in Table 2.

| **Algorithm** | **Recommended Parameters** |
|---|---|
| ML-KEM | ML-KEM-768, ML-KEM-1024 |
| FrodoKEM | FrodoKEM-976, FrodoKEM-1344 |
| ML-DSA | ML-DSA-65, ML-DSA-87 |
| SLH-DSA | Security Level 3 and Level 5 parameter sets |

Table 2: ECCG recommended post-quantum parameter sets.

The ECCG recommends the use of higher parameter sets such as ML-KEM-768 or ML-KEM-1024, FrodoKEM-976 or FrodoKEM-1344, and ML-DSA-65 or ML-DSA-87, which correspond to higher security levels.

### 4.3 Implications for Our Implementation

Based on the ECCG recommendations, we selected **ML-KEM-768** as the post-quantum component for our hybrid TLS implementation. Since ML-KEM is a lattice-based mechanism, the ECCG document requires it to be used in hybrid mode rather than standalone. Combined with X25519, the hybrid group `X25519MLKEM768` satisfies this requirement. The classical component (X25519) ensures backward compatibility and well-understood security, while ML-KEM-768 provides quantum resistance at one of the recommended parameter levels.

## 5 Implementation

In order to evaluate the performance impact of hybrid cryptography, we implemented a TLS 1.3 client and server in C using the OpenSSL library.

### 5.1 Software Environment

| **Component** | **Version** |
|---|---|
| Operating system | Ubuntu 24.04.2 LTS |
| OpenSSL | 3.5.0 (built from source) |
| Compiler | GCC (via `build-essential`) |
| Interface | Linux loopback (127.0.0.1) |

Table 3: Software environment used for the implementation and benchmarks.

Ubuntu 24.04 ships OpenSSL 3.0.x, which does not support hybrid key exchange groups. We therefore compiled OpenSSL 3.5.0 from source with hybrid KEM support enabled. A Dockerfile is provided in the repository to reproduce the environment.

### 5.2 TLS Client and Server

The **server** program creates a TCP socket on the loopback interface (port 4433), loads an ECDSA P-256 self-signed certificate, and accepts TLS 1.3 connections. For each connection, it performs the TLS handshake, reads data sent by the client, sends a short response, and cleanly shuts down the TLS session. The server supports accepting multiple consecutive connections via the `-n` flag.

The **client** program connects to the server and performs the TLS 1.3 handshake. After the handshake completes, it sends 4096 bytes of application data and reads the server response. The client collects all performance measurements and reports them either in human-readable or CSV format.

### 5.3 Measurement Instrumentation

Three metrics are measured for each TLS session:

1. **Handshake time:** The wall-clock duration of `SSL_connect()`, measured using `clock_gettime(CLOCK_MO`
   This captures the full TLS 1.3 handshake including key exchange and authentication.
2. **Data transfer time:** The wall-clock time to send 4096 bytes via `SSL_write()` and receive the server response via `SSL_read()`, measured after the handshake is complete.
3. **Handshake bytes:** The total number of TCP bytes exchanged during the handshake. This is measured using a custom *counting BIO filter* that wraps the socket BIO. The filter intercepts all `BIO_read()` and `BIO_write()` calls and accumulates byte counts, which are reset before each handshake.

The counting BIO approach measures raw bytes on the wire, including TLS record headers, and provides an accurate picture of the communication overhead introduced by different key exchange mechanisms.

### 5.4 TLS Configurations

Two configurations are tested, differing only in the key exchange group:

| Configuration | Key Exchange | Authentication | Cipher Suite |
|---|---|---|---|
| Classical | X25519 (ECDHE) | ECDSA P-256 | TLS_AES_256_GCM_SHA384 |
| Hybrid | X25519MLKEM768 | ECDSA P-256 | TLS_AES_256_GCM_SHA384 |

Table 4: TLS configurations used in the experiments.

The hybrid configuration uses the concatenation-based combiner approach: X25519 and ML-KEM-768 key shares are exchanged independently during the TLS handshake, and OpenSSL's TLS 1.3 key schedule (HKDF) combines the resulting shared secrets. Authentication uses classical ECDSA in both configurations, as hybrid signature schemes are not yet supported in TLS 1.3 by OpenSSL.

## 6 Benchmark Methodology

### 6.1 Experimental Setup

All experiments run on the Linux loopback interface (`127.0.0.1`), eliminating network variability and isolating the cryptographic performance differences between configurations. The server and client run as separate processes on the same machine inside a Docker container running Ubuntu 24.04.

### 6.2 Metrics

Each TLS session produces the following measurements:

- **Handshake time (ms)** – duration of `SSL_connect()`.
- **Data transfer time (ms)** – duration of `SSL_write()` + `SSL_read()` after the handshake.
- **Handshake bytes sent/received** – TCP bytes exchanged during the handshake, as counted by the BIO filter.

### 6.3 Measurement Procedure

To reduce noise, each configuration is run $N$ times within a single server–client process pair. The server accepts $N$ consecutive connections and the client connects $N$ times, reusing the same `SSL_CTX` (but creating a new `SSL` session each time). This avoids process startup overhead between runs.

Results are output in CSV format. The benchmark script computes the **mean** and **standard deviation** of handshake time, data transfer time, and handshake byte count across all runs for each configuration.

## 7 Results

Table 5 presents the benchmark results averaged over 10 runs for each configuration.

| Configuration | Handshake Time (ms) | Handshake Bytes | Transfer Time (ms) |
|---|---|---|---|
| Classical (X25519) | 1.42 ± 0.94 | 1054 | 0.35 ± 0.42 |
| Hybrid (X25519MLKEM768) | 2.55 ± 0.51 | 3318 | 0.34 ± 0.10 |

Table 5: Benchmark results: mean ± standard deviation over 10 runs.

Key observations:

- The hybrid handshake is approximately **1.8× slower** than the classical handshake (2.55 ms vs. 1.42 ms).
- The hybrid handshake exchanges approximately **3.1× more bytes** than the classical handshake (3318 vs. 1054 bytes). This increase is due to the additional ML-KEM-768 key material exchanged during the handshake, as observed in our measurements.
- **Data transfer time is unaffected** by the choice of key exchange, as both configurations negotiate the same symmetric cipher suite (TLS_AES_256_GCM_SHA384). After the handshake, both configurations behave identically.

## 8 Discussion

### 8.1 Performance Overhead

The hybrid configuration introduces a measurable increase in handshake time and communication overhead. However, the absolute increase is small: roughly 1 ms of additional handshake latency and approximately 2.3 KB of additional data. On modern networks, this overhead is unlikely to be noticeable for most applications.

The handshake time increase comes from two sources: (1) the additional computation needed for ML-KEM-768 key generation and decapsulation, and (2) the larger TLS messages that must

be transmitted. On the loopback interface, transmission time is negligible, so the measured overhead is primarily computational.

### 8.2 Communication Overhead

The hybrid handshake is larger because additional ML-KEM-768 key material (public key and ciphertext) must be exchanged alongside the X25519 key shares. In networks with low bandwidth or high latency, this additional data could increase handshake time further. The optional network simulation script (`net_sim.sh`) using `tc` can be used to study this effect.

### 8.3 Practical Deployment Considerations

Despite the overhead, hybrid key exchange is practical for real-world deployment:

- Major browsers (Chrome, Firefox) and services (Cloudflare) have already deployed X25519MLKEM768 in production.
- The overhead is a one-time cost per connection; long-lived connections amortize it completely.
- The additional latency is smaller than typical network round-trip times (often 10–100 ms), making it negligible in practice.
- Using hybrid mode during the transition period follows the ECCG recommendation and provides protection against "harvest now, decrypt later" attacks, where an adversary records encrypted traffic today to decrypt it with a future quantum computer.

### 8.4 Limitations

Our experiments have some limitations:

- Benchmarks run on the loopback interface, which eliminates network effects. Real-world performance would include network latency and potential fragmentation of larger handshake messages.
- Only one hybrid combination (X25519MLKEM768) was tested. Other combinations (e.g., with ML-KEM-1024) would show different trade-offs.
- Authentication uses classical ECDSA in both configurations, as hybrid signatures are not yet supported in OpenSSL for TLS. A fully hybrid deployment would also need hybrid authentication.

## 9 Conclusion

This project investigated hybrid cryptography in TLS 1.3 by reviewing ETSI deployment considerations for hybrid constructions and ECCG/ENISA recommendations, and by implementing and benchmarking a hybrid TLS deployment using OpenSSL 3.5.

Our experiments show that hybrid key exchange (X25519MLKEM768) introduces approximately 1.8× the handshake latency and 3.1× the handshake data compared to classical key exchange (X25519). However, the absolute overhead is small (approximately 1 ms and 2.3 KB), and data transfer after the handshake is unaffected.

These results support the ECCG recommendation to deploy hybrid cryptography during the transition to post-quantum security: the performance cost is modest, while the security benefit — protection against both classical and quantum adversaries — is significant.

## Individual Contributions

| Student | Contribution |
|---|---|
| Kristofer Birgir Hjörleifsson | Implementation, benchmarking |
| Brynja Eyfjörð Jónsdóttir | Literature review, report writing |
| Karítas Ósk Pálmadóttir | Literature review, report writing |

## References


[1] ETSI, "Quantum-Safe Cryptography (QSC); Deployment Considerations for Hybrid Schemes," ETSI TR 103 966 V1.1.1, October 2024.

[2] ECCG, "Agreed Cryptographic Mechanisms," Version 2, European Cybersecurity Certification Group (ECCG), ENISA, 2024. https://certification.enisa.europa.eu/document/download/a845662b-aee0-484e-9191-890c4cfa7aaa_en

[3] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, August 2018. https://doi.org/10.17487/RFC8446

[4] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, August 2024.

[5] NIST, "Module-Lattice-Based Digital Signature Standard," FIPS 204, August 2024.

[6] NIST, "Stateless Hash-Based Digital Signature Standard," FIPS 205, August 2024.

[7] OpenSSL Project, "OpenSSL 3.5.0 Release," April 2025. https://www.openssl.org/

[8] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid Key Exchange in TLS 1.3," Internet-Draft, IETF, 2024.